

Iteration 4

Alex Kramer, Kreena Totala

March 2026

1 Purpose of the Methodology

Our methodology combines text-driven signal extraction with multi-criteria ranking to translate an Airbnb listing into a musically coherent playlist. The Spotify-side pipeline (K-means clustering over audio features, zero-shot emotion inference, lyric keyword matching, and retrieve-then-aggregate audio targets) captures complementary aspects of “vibe.” Comparing multiple signals within the same scoring function is necessary because no single modality fully captures the experience described in a listing: lyrics encode place-specific cues, emotion inference captures mood, audio cosine similarity captures timbre/energy, and cluster affinity enforces global coherence. Evaluating how these layers interact reveals which signals dominate for a given listing and guides future adjustments to weights or model components.

2 Problem Statement

2.1 Defining the Problem

This project is a hybrid retrieval and ranking problem. It combines:

- **Unsupervised clustering** (K-means) over audio feature space to discover latent track groupings.
- **Semantic retrieval** using dense sentence embeddings (retrieve-then-aggregate) to map natural language keywords to audio feature targets.
- **Zero-shot text classification** (NLI) to infer emotional intent from keywords without labeled training data.
- **Multi-criteria ranking** combining heterogeneous scores into a single playlist ordering.

There is no single ground-truth label; the problem is framed as a relevance ranking task where the goal is to maximize alignment between the listing’s expressed mood and the recommended tracks across multiple complementary signals.

2.2 Significance

Music recommendation is a well-studied domain, but cross-modal recommendation, or deriving music preferences from non-musical context, remains relatively underexplored. Existing systems (Spotify, Apple Music) rely on explicit user history or collaborative filtering. This project demonstrates that rich natural language descriptions of *places and experiences* can serve as a viable recommendation signal, with applications in:

- **Travel and hospitality:** curated ambient music for Airbnb stays.
- **Contextual recommendation:** extending playlist generation to hotel lobbies, retail environments, or event venues.
- **Cold-start scenarios:** generating playlists for new users with no listening history, using only a location or experience description.

3 Data Collection and Preparation

3.1 Data Sources

Spotify Dataset The Spotify track catalog is sourced from a publicly available Kaggle dataset: *Spotify Dataset with Lyrics* (551,443 tracks). The dataset includes audio features extracted via the Spotify Web API, track metadata, pre-labeled emotion tags, genre annotations, and song lyrics.

Airbnb Dataset This project uses publicly available Airbnb listing data from *Inside Airbnb* to enable reproducible analysis without the need for web scraping. Inside Airbnb provides detailed, city-level datasets containing rich listing information such as titles, descriptions, and neighborhood overviews. These natural-language fields are particularly valuable for extracting contextual signals about each stay, allowing us to capture the atmosphere, location characteristics, and overall experience associated with different listings.

To support transparency and reproducibility, the specific city-level subsets used in this project will be uploaded to our GitHub repository. This ensures that all experiments can be replicated and extended using the exact same data.

3.2 Data Description

Spotify Dataset. The raw dataset contains 551,443 rows and 39 columns. After dropping uninformative feature groups (columns prefixed with “Good for”, “Similar Artist”, “Similar Song”, “Similarity Score”), renaming columns to clean snake_case identifiers, and parsing non-standard formats, the working dataset retains 21 columns. Key fields used in the recommendation pipeline are:

Field	Type	Description
track_name	string	Track title
artist	string	Artist name(s)
lyrics	string	Full song lyrics
emotion	categorical	Pre-labeled emotion tag (joy, sadness, anger, fear, surprise, neutral)
genre	categorical	Genre label
popularity	int	Spotify popularity score (0–100)
danceability	float	Rhythmic suitability for dancing (0–1)
energy	float	Perceptual intensity and activity (0–1)
valence	float	Musical positiveness (0–1)
tempo	float	Beats per minute
loudness	float	Average loudness in dB
duration_s	int	Track duration in seconds

Table 1: Key Spotify dataset fields used in the recommendation pipeline.

Airbnb Dataset The raw dataset is constructed from multiple city-level subsets obtained from Inside Airbnb, which we combine to create a more diverse set of listings for experimentation. The combined dataset contains 67,426 rows, and 80 columns. After removing uninformative features we retained 10 column. Key fields used in the Airbnb NLP Pipeline are:

Field	Type	Description
id	int	Unique listing ID
listing_url	string	Listing URL
name	string	Name (or Title) of listing
description	string	Listing description
neighborhood_overview	string	Neighborhood description for listing
neighborhood_cleansed	string	Cleaned neighborhood (listing location)
property_type	string	Listing property type (home, shared house, apt., etc.)
room_type	string	Listing room type
accommodates	int	Number of people listing accommodates
amenities	string	Listing amenities (Wi-Fi, parking, washer, dryer, etc.)

Table 2: Key Inside Airbnb dataset fields used in the NLP pipeline.

3.3 Preprocessing Steps

Spotify Preprocessing

1. **Column cleanup:** Drop “Good for *”, “Similar Artist/Song/Score *” columns (all near-zero variance or redundant).

2. **Column renaming:** Standardize to snake_case (e.g., Loudness (db) → loudness, Positiveness → valence).
3. **Loudness parsing:** Strip the “db” suffix and cast to float (e.g., "-6.85db" → -6.85).
4. **Duration parsing:** Convert MM:SS string format to integer seconds.
5. **Missing value removal:** Drop rows missing any of the 11 audio feature columns.
6. **Outlier removal:** Remove tracks with any audio feature z -score exceeding ± 4.0 (flags $\approx 0.3\%$ of tracks).
7. **Feature scaling:** Apply `StandardScaler` ($\mu = 0$, $\sigma = 1$) to all 11 audio feature columns to produce the scaled feature matrix used for clustering and similarity computation.
8. **Dataset filtering:** For deployment, the dataset is filtered to the top 40,000 tracks by popularity score (range 63–100) to meet GitHub file size constraints while preserving recommendation quality.

Airbnb Preprocessing

1. **Column cleanup:** Drop the following columns:
 - scrape_id, last_scraped, source, picture_url, host_id
 - host_url, host_name, host_since, host_location, host_about
 - host_response_time, host_response_rate, host_acceptance_rate
 - host_is_superhost, host_thumbnail_url, host_picture_url
 - host_neighbourhood, host_listings_count, host_total_listings_count
 - host_verifications, host_has_profile_pic, host_identity_verified
 - neighbourhood, neighbourhood_group_cleansed, latitude, longitude
 - bathrooms, bathrooms_text, bedrooms, beds, price
 - minimum_nights, maximum_nights, minimum_minimum_nights
 - maximum_minimum_nights, minimum_maximum_nights
 - maximum_maximum_nights

This step reduces dimensionality and removes redundant, highly sparse, or irrelevant metadata not required for downstream analysis. Retained columns (e.g., `id`, `listing_url`, `name`, `description`, `neighbourhood_overview`, `neighbourhood_cleansed`, `amenities`, `room_type`, and `property_type`) were selected to preserve semantically rich textual information and key contextual attributes. These features are particularly relevant for downstream natural language processing tasks, including keyword extraction and emotion (sentiment) analysis of listing descriptions.

2. **Joining city-level datasets:** Individual city-level Airbnb extracts are combined into a single dataset through row-wise concatenation after schema alignment. Columns are conditionally selected to ensure consistency between files (cities), allowing for the integration of datasets with heterogeneous structures while preserving shared attributes.
3. **Missing value removal:** Remove observations with missing or null values in key textual fields (e.g., `description`).
4. **Text clean-up:** Apply standard natural language preprocessing techniques, including removal of non-ASCII characters, stripping of HTML tags, elimination of punctuation, lowercasing all text, and normalization of whitespace. This step standardizes textual input and improves downstream processing, such as tokenization and feature extraction.

4 Model Selection

4.1 Models Considered

Clustering. K-means was selected over alternatives (DBSCAN, hierarchical clustering, Gaussian Mixture Models) for the following reasons:

- **Scalability:** K-means scales to 500k+ tracks in seconds; DBSCAN and hierarchical methods are computationally prohibitive at this scale.
- **Interpretability:** Cluster centroids are directly interpretable as “average audio profiles”, enabling the cluster boost layer to identify the most acoustically relevant cluster for a given query.
- **Elbow method validation:** Silhouette scores and the elbow criterion on within-cluster sum of squares (WCSS) were computed for $k \in [2, 12]$, with $k = 5$ selected as the inflection point.

Keyword-to-Emotion Mapping. Three approaches were considered:

1. **Rule-based lexicon matching** (NRC Emotion Lexicon): fast, no model required, but limited to single-word lookup with no semantic generalization.
2. **Sentence embedding cosine similarity to emotion anchors:** embeds keywords and computes similarity to fixed emotion anchor phrases. Simple but sensitive to anchor phrasing.
3. **Zero-shot NLI classification** (facebook/bart-large-mnli): classifies concatenated keywords against emotion labels without any task-specific training. Selected for its robustness to novel phrasing and strong out-of-the-box performance on short text classification.

Zero-shot NLI was selected for the emotion layer because it handles compositional keyword strings (e.g., “ocean breeze, relaxed, sunny terrace”) as a unified passage, capturing inter-keyword context that lexicon lookup cannot.

Audio Target Construction. To convert natural language keywords into an audio feature target vector, a **retrieve-then-aggregate** strategy was used:

1. Embed all keywords using **all-MiniLM-L6-v2** (sentence-transformers).
2. Compute cosine similarity between the mean keyword embedding and all pre-embedded track lyrics.
3. Average the scaled audio features of the top- k ($k = 20$) most similar tracks.

This produces a semantically grounded audio target vector directly in the scaled feature space, avoiding the need for manual feature-to-keyword mappings.

4.2 Final Model Selection

The final system uses a **4-layer weighted scoring pipeline**:

$$\text{score}(t) = w_1 \cdot s_{\text{lyrics}}(t) + w_2 \cdot s_{\text{emotion}}(t) + w_3 \cdot s_{\text{audio}}(t) + w_4 \cdot s_{\text{cluster}}(t) \quad (1)$$

with default weights $w_1 = 0.35$, $w_2 = 0.25$, $w_3 = 0.25$, $w_4 = 0.15$.

The weights reflect the relative reliability of each signal: lyrics matching is the most direct signal for location-specific atmosphere, emotion and audio similarity are complementary semantic signals, and cluster membership is a soft structural prior.

5 Model Development and Training

5.1 Architecture

The recommendation pipeline is stateless at inference time (no gradient-based training). The components are:

1. **Lyrics layer** (s_{lyrics}): Regular expression matching of location/place terms (identified as keywords with low NLI emotion confidence, < 0.30) against lowercase track lyrics. Score is the clipped count of matched terms, normalized to $[0, 1]$.
2. **Emotion layer** (s_{emotion}): The dominant emotion(s) inferred by NLI are matched against each track’s pre-labeled **emotion** column. The top emotion scores 1.0, second 0.5, all others 0.25. Score is clipped to $[0, 1]$.

3. **Audio cosine layer** (s_{audio}): Cosine similarity between each track’s z -scored 11-dimensional audio feature vector and the retrieve-then-aggregate target vector. Raw cosine similarity $\in [-1, 1]$ is rescaled to $[0, 1]$ as $\frac{\text{sim}+1}{2}$.
4. **Cluster boost layer** (s_{cluster}): The K-means cluster whose centroid minimizes L2 distance to the audio target vector is identified. Tracks in that cluster receive $s_{\text{cluster}} = 1.0$; all others receive 0.0.

5.2 K-means Clustering Configuration

- **Input features:** 11 z -scored audio features (danceability, energy, loudness, speechiness, acousticness, instrumentality, liveness, valence, tempo, duration_s, popularity).
- **Number of clusters:** $k = 5$, selected via elbow method on WCSS and silhouette score analysis.
- **Initialization:** k-means++ for stable centroid initialization.
- **Convergence:** Standard sklearn defaults (max 300 iterations, tolerance 10^{-4}).

5.3 Embedding Models

- **Sentence encoder:** all-MiniLM-L6-v2 (384-dimensional embeddings, normalized). Used for both lyric indexing and keyword embedding.
- **Zero-shot classifier:** facebook/bart-large-mnli. Applied to the concatenated keyword string at query time.
- Both models are loaded once at inference time and cached in memory.

5.4 Training / Index Building

There is no gradient-based training in this system. The offline preparation steps are:

1. Fit K-means on the full scaled feature matrix.
2. Pre-embed all track lyrics with all-MiniLM-L6-v2 (batch size 256) to build the retrieval index used by the audio target construction step.

Both steps run once at application startup via `@st.cache_resource` and persist for the lifetime of the server process.

Parameter	Value	Rationale
k (clusters)	5	Elbow/silhouette analysis
Retrieve-then-aggregate k	20	Balances specificity vs. noise in audio target
Outlier z -threshold	4.0	Removes extreme outliers while retaining edge-case tracks
Layer weights	0.35 / 0.25 / 0.25 / 0.15	Reflect relative signal reliability

Table 3: Key hyperparameters and selection rationale.

5.5 Hyperparameter Decisions

6 Evaluation and Comparison

6.1 Proposed Evaluation Metrics

Since this is a retrieval/ranking system with no labeled ground truth, evaluation focuses on proxy metrics computable from the existing data:

1. **Intra-playlist audio diversity:** Mean pairwise cosine *distance* ($1 - \text{similarity}$) between all pairs of track audio feature vectors in the playlist. A score near 0 indicates all tracks are acoustically identical; near 1 indicates maximal diversity. Prevents the recommender from collapsing to a single cluster.
2. **Genre entropy:** Shannon entropy $H = -\sum_g p_g \log p_g$ over the genre distribution in the playlist. Higher entropy indicates broader genre coverage.
3. **Layer contribution balance:** Mean per-layer weighted score contribution across the playlist. A balanced system uses all four layers; dominance of any single layer ($>60\%$ of total) suggests the others are not adding value.

6.2 Baseline Comparisons

- **Popularity-only baseline:** Top- N tracks by Spotify popularity score, ignoring all keyword signals. Tests whether the NLP layers add value over simple popularity ranking.

The recommendation system is expected to significantly outperform baseline popularity ranking. Intra-playlist diversity may be lower than random (since the system intentionally targets a specific mood), which is expected and acceptable.

Appendix: System Architecture Overview

